ProPosition Tool Documentation

Overview

ProPosition is a Flask-based location visualization tool. It accepts customer or site coordinates through an uploaded Excel/CSV file or a manual latitude/longitude form, enriches each coordinate using Google Maps services, generates visual location assets, and presents the results in a browser-based report.

The tool currently runs as a web application with in-memory background jobs. It does not write generated reports to disk. Instead, it stores job state, report summaries, and ZIP output bytes in process memory while the application is running.

Primary Capabilities

- Upload a CSV, XLS, or XLSX file containing latitude and longitude columns.
- Manually enter a single latitude and longitude pair.
- Reverse geocode each coordinate into an address.
- Retrieve Google Place details for the resolved location.
- Generate a static map image for each coordinate.
- Generate a Street View image when available.
- Generate a lightweight 360-degree Street View HTML page.
- Display processing progress in the browser.
- Render a results report with tabs for summary, static maps, Street View images, and 360-degree views.
- Download all generated assets as a ZIP archive.

Repository Structure

```
.
|-- app.py
|-- maps2.py
|-- requirements.txt
|-- DOCUMENTATION.md
|-- .gitignore
|-- static/
|   `-- img/
```

```
|       `-- protiviti_logo.png
`-- templates/
    |-- index.html
    |-- progress.html
    `-- report.html
```

File Responsibilities

app.py

`app.py` contains the Flask application.

Responsibilities:

- Loads `GOOGLE_MAPS_API_KEY` from environment variables using `python-dotenv`.
- Defines Flask routes.
- Handles file uploads and manual coordinate submissions.
- Starts background processing threads.
- Stores job status and results in the global `job_data` dictionary.
- Builds an in-memory ZIP file from generated outputs.
- Serves generated files from the in-memory ZIP archive.
- Renders the upload, progress, and report templates.

Important globals:

```
API_KEY = os.getenv("GOOGLE_MAPS_API_KEY")
job_data = {}
```

maps2.py

`maps2.py` contains the Google Maps processing logic.

Responsibilities:

- Normalizes input dataframe columns.
- Calls Google Geocoding API.
- Calls Google Place Details API.
- Calls Google Static Maps API.
- Calls Google Street View API.
- Generates 360-degree Street View HTML content.
- Processes dataframe rows concurrently with `ThreadPoolExecutor`.

- Returns summary rows and generated file objects to `app.py`.

templates/index.html

The landing page for the tool.

Provides:

- File upload input for Excel/CSV.
- Manual latitude field.
- Manual longitude field.
- Submit button.
- Flash message display.

templates/progress.html

The progress page shown while a job is running.

Provides:

- Bootstrap progress bar.
- Polling JavaScript that calls `/progress_api/<job>` once per second.
- Automatic redirect to `/report/<job>` when processing is complete.

templates/report.html

The final results page.

Provides:

- Summary table.
- Static Maps tab.
- Street View tab.
- Street 360 tab.
- ZIP download button.
- Link back to upload another file.

requirements.txt

Python package dependencies:

```
gunicorn
Flask
```

```
pandas
openpyxl
python-dotenv
pillow
requests
```

Runtime Requirements

Python

Use a Python version compatible with Flask, pandas, openpyxl, Pillow, and requests. Python 3.9 or newer is recommended.

Google Maps API Key

The application requires this environment variable:

```
GOOGLE_MAPS_API_KEY
```

The key must have access to the Google APIs used by this tool:

- Geocoding API
- Places API
- Maps Static API
- Street View Static API
- Maps JavaScript API

Recommended API key restrictions:

- Restrict the key to only the APIs listed above.
- Use HTTP referrer restrictions for browser-rendered Maps JavaScript usage.
- Use separate server-side and browser-side keys for production if possible.
- Monitor usage and billing in Google Cloud Console.

Local Setup

1. Create a virtual environment

```
python -m venv .venv
```

2. Activate the virtual environment

On Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

On macOS/Linux:

```
source .venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Create a `.env` file

Create a file named `.env` in the project root:

```
GOOGLE_MAPS_API_KEY=your_google_maps_api_key_here
```

The `.gitignore` file already excludes `.env`.

5. Run the app

```
python app.py
```

By default, Flask starts at:

```
http://127.0.0.1:5000
```

Production Startup

`requirements.txt` includes `gunicorn`, which is typically used on Linux-based deployments:

```
gunicorn app:app
```

Important production note: the current app stores jobs in process memory. Running multiple Gunicorn workers can cause inconsistent behavior because each worker has its own separate `job_data` dictionary. For production, use a shared backend such as Redis, a database, or persistent object storage for job state and generated files.

Input Formats

CSV Input

CSV files are read with:

```
pd.read_csv(BytesIO(f.read()))
```

Accepted file extension:

```
.csv
```

Excel Input

Excel files are read with:

```
pd.read_excel(BytesIO(f.read()))
```

Accepted file extensions:

```
.xls
.xlsx
```

Required Coordinate Columns

The tool accepts flexible column names and normalizes them internally.

Latitude can be provided as:

```
lat
latitude
```

Longitude can be provided as:

```
lng
long
longitude
```

Column matching is case-insensitive because the implementation lowercases column names before matching.

Example CSV:

```
lat,lng
19.0760,72.8777
28.6139,77.2090
12.9716,77.5946
```

Example CSV with alternate names:

```
Latitude,Longitude
19.0760,72.8777
28.6139,77.2090
```

Manual Coordinate Input

The upload page also accepts one coordinate pair:

- Latitude
- Longitude

These values are converted to `float` before processing.

Request Flow

1. User opens `/`

The `index()` route renders `templates/index.html`.

2. User submits a file or coordinates

The `index()` route receives the POST request.

File upload path:

- Reads uploaded file.
- Validates extension.
- Loads file into a pandas dataframe.
- Creates a job name from the filename and current UTC timestamp.

Manual coordinate path:

- Parses latitude and longitude as floats.
- Creates a one-row pandas dataframe.
- Creates a job name using the `manual_` prefix and current UTC timestamp.

3. App starts a background thread

`app.py` starts:

```
threading.Thread(target=run_processing, args=(job_name, df))
```

The browser is redirected to:

```
/progress/<job>
```

4. Browser polls progress API

`templates/progress.html` calls:

```
/progress_api/<job>
```

The API returns:

```
{
  "progress": "1/10",
  "status": "Processing location 1/10..."
}
```

When processing completes, the API returns:

```
{
  "progress": "done",
```

```
  "status": "Complete"
}
```

The browser then redirects to:

```
/report/<job>
```

5. Report page renders results

The `report()` route reads the job summary from memory and renders `templates/report.html`.

6. User downloads the ZIP

The `download_all()` route serves the in-memory ZIP archive:

```
/download_all/<job>
```

Flask Routes

`GET /`

Renders the upload/manual coordinate page.

Template:

```
templates/index.html
```

`POST /`

Accepts either:

- Uploaded file in form field `file`
- Manual coordinate fields `lat` and `lng`

On success, starts background processing and redirects to `/progress/<job>`.

On validation failure, flashes an error and redirects back to `/`.

GET /progress/<job>

Renders the progress page for a job.

Template:

```
templates/progress.html
```

GET /progress_api/<job>

Returns current job progress as JSON.

Example:

```
{
  "progress": "3/20",
  "status": "Processing location 3/20..."
}
```

If the job does not exist:

```
{
  "progress": "0/1",
  "status": "Job not found"
}
```

GET /report/<job>

Renders the completed report.

Template:

```
templates/report.html
```

Returns HTTP 404 if the job does not exist or the summary is not available.

GET /download_all/<job>

Downloads the generated ZIP archive.

Response type:

```
application/zip
```

Returns HTTP 404 if the archive is not available.

`GET /outputs/<job>/<path:filename>`

Serves generated files from the in-memory ZIP archive.

Supported content types:

- `.png` : `image/png`
- `.jpg` or `.jpeg` : `image/jpeg`
- `.html` : `text/html`
- other: `application/octet-stream`

Processing Details

Column Normalization

`maps2._normalize_cols()` maps latitude and longitude aliases to canonical columns:

```
lat
lng
```

It returns a dataframe containing only:

```
df[["lat", "lng"]]
```

If a required column is missing, it raises:

```
ValueError: Missing column: lat
```

or:

```
ValueError: Missing column: lng
```

Per-Location Processing

For each coordinate pair, the tool:

1. Calls reverse geocoding.
2. Uses the returned `place_id` to fetch place details.
3. Requests a static map image.
4. Requests a Street View image.
5. Generates 360-degree HTML using the Maps JavaScript API.
6. Adds summary metadata and file references to the result row.

Concurrent Execution

`maps2.process_dataframe()` uses:

```
ThreadPoolExecutor(max_workers=10)
```

This means up to 10 locations may be processed at the same time.

Concurrency improves throughput, but it can also increase Google API usage quickly. For large uploads, review Google API quotas and billing limits.

Google API Calls

Reverse Geocoding

Function:

```
reverse_geocode(lat, lng)
```

Endpoint:

```
https://maps.googleapis.com/maps/api/geocode/json
```

Parameters:

- `latlng`
- `key`

Returns:

- formatted address
- place ID

Place Details

Function:

```
place_details(place_id)
```

Endpoint:

```
https://maps.googleapis.com/maps/api/place/details/json
```

Requested fields:

```
name,types,business_status,formatted_address
```

Returns a Google Place result dictionary.

Static Map

Function:

```
get_static_map(lat, lng)
```

Endpoint:

```
https://maps.googleapis.com/maps/api/staticmap
```

Default settings:

```
STATIC_SIZE = "640x400"
STATIC_ZOOM = 17
maptype = "roadmap"
marker = red marker at coordinate
```

Output:

```
map.png
```

Street View Static Image

Function:

```
get_street_view(lat, lng)
```

Endpoint:

```
https://maps.googleapis.com/maps/api/streetview
```

Default settings:

```
STREET_SIZE = "640x400"
STREET_HEADING = 0
STREET_PITCH = 0
```

Output:

```
street.jpg
```

If no valid Street View image is available, the current implementation returns `None` only when the HTTP status is not 200 or the response content is empty.

Street View 360 HTML

Function:

```
get_360_html(lat, lng)
```

Output:

```
360.html
```

The HTML embeds the Maps JavaScript API and initializes:

```
new google.maps.StreetViewPanorama(...)
```

Important: this generated HTML includes the Google Maps API key in the script URL.

Generated Result Row

Successful result rows contain:

```
{
    "lat": lat,
    "lng": lng,
    "address": "...",
    "place_name": "...",
    "business_status": "...",
    "types": "...",
    "static_map": "1/map.png",
    "street_img": "1/street.jpg",
    "street360": "1/360.html",
    "status": "ok"
}
```

Error result rows contain:

```
{
    "lat": lat,
    "lng": lng,
    "address": "",
    "status": "error",
    "note": "...",
    "static_map": "",
    "street_img": "",
    "street360": ""
}
```

ZIP Output Structure

Generated files are grouped by row number.

Example:

```
1/map.png
1/street.jpg
1/360.html
2/map.png
2/street.jpg
2/360.html
```

If Street View is not available, `street.jpg` may be omitted.

Report UI

The report page has four tabs.

Summary

Displays a table with:

- row number
- address

Static Maps

Displays `map.png` thumbnails for each row.

Street View

Displays `street.jpg` thumbnails when available.

If no Street View image is available, the UI shows:

```
No view
```

Street 360

Displays each generated `360.html` file in an iframe.

Job Storage Model

The app uses an in-memory dictionary:

```
job_data = {
    job_name: {
        "progress": "0/1",
        "status": "Initializing",
        "results": None,
        "zip_data": None,
        "summary": None
    }
}
```

This design is simple and works for local demos, but it has important limitations:

- Jobs disappear when the app restarts.
- Jobs are not shared across multiple worker processes.
- Memory usage grows as more jobs are processed.
- There is no job cleanup or expiration.
- Large uploads can consume significant RAM.

Error Handling

Current behavior:

- Unsupported file types flash `Unsupported file type.`
- Invalid manual coordinates flash `Invalid coordinates.`
- Missing input flashes `Upload a file or enter coordinates.`
- Reverse geocoding and place details errors become per-row error results.
- Some downstream errors, such as static map request failures, can currently terminate the background thread.

Recommended hardening:

- Add a top-level `try/except` in `run_processing()`.
- Store failed job state in `job_data`.
- Update the progress API to expose failure states.
- Update `progress.html` to stop polling and show errors.
- Catch per-row image generation errors in `maps2.process_dataframe()`.

Security Considerations

API Key Exposure

The generated 360-degree HTML includes:

```
<script src="https://maps.googleapis.com/maps/api/js?key=..."></script>
```

This means the key can be visible to users who download or inspect generated HTML files.

Recommended mitigation:

- Use separate server-side and browser-side Google API keys.
- Restrict the browser-side key by HTTP referrer.
- Restrict the server-side key by API and, where possible, by IP.
- Avoid including unrestricted keys in downloadable files.

Upload Validation

Current validation checks only file extension:

```
.csv
.xls
.xlsx
```

Recommended improvements:

- Configure `MAX_CONTENT_LENGTH` in Flask.
- Validate file content type where possible.
- Limit number of rows per upload.
- Validate coordinate ranges.
- Return user-friendly errors for malformed files.

Job Access

Job URLs are based on filename/manual prefix plus timestamp. Anyone with the URL can access the job output while it is in memory.

Recommended improvements:

- Use random unguessable job IDs.
- Add user authentication if needed.
- Add authorization checks before serving reports and output files.

Debug Mode

The app currently uses:

```
app.run(debug=True)
```

Do not run debug mode in production.

Known Limitations

- Generated output is not persisted to disk or cloud storage.
- In-memory job storage is not suitable for multi-worker production deployments.
- Background threads are started directly from Flask request handlers.
- There is no retry logic for transient Google API failures.
- There is no rate limiting.
- There is no row limit.
- There is no upload size limit.
- Job IDs are predictable.
- Output file serving uses suffix matching instead of exact path matching.
- Result row order may differ from upload row order because processing uses `as_completed()`.
- Some template text currently appears with encoding artifacts in place of dashes, arrows, and degree symbols.
- The footer year is hard-coded.
- `PIL.Image` is imported but not used.

Recommended Production Architecture

For a production version, replace the current in-memory/threaded design with:

- Flask or another web framework for the UI.
- Redis or database-backed job state.
- Celery, RQ, Dramatiq, or another worker queue for background processing.
- Object storage or filesystem storage for generated images and ZIP files.
- Random UUIDs for job IDs.
- Authentication and authorization if reports contain sensitive customer/site data.
- Centralized logging and error reporting.
- Explicit Google API quota handling and retry policy.

Example production flow:

```
User upload
  -> Flask validates input
  -> Job record is created in database
  -> Background task is queued
  -> Worker processes rows
  -> Generated files are written to object storage
  -> Job progress is updated in Redis/database
  -> Browser polls progress endpoint
  -> Report reads completed artifacts from storage
```

Troubleshooting

`RuntimeError: GOOGLE_MAPS_API_KEY not set.`

Cause:

The environment variable is missing.

Fix:

Create a `.env` file:

```
GOOGLE_MAPS_API_KEY=your_google_maps_api_key_here
```

Then restart the app.

`Unsupported file type.`

Cause:

The uploaded file extension is not one of:

```
csv, xls, xlsx
```

Fix:

Upload a supported file format.

`Missing column: lat` or `Missing column: lng`

Cause:

The uploaded file does not contain recognizable coordinate columns.

Fix:

Use one of the accepted latitude names:

```
lat
latitude
```

Use one of the accepted longitude names:

```
lng
long
longitude
```

Progress page never completes

Possible causes:

- The background thread crashed.
- A Google API request failed after reverse geocoding.
- The app restarted during processing.
- The job was created in one process but the progress request hit another process.

Fix:

- Check server logs.
- Run with a single worker during local testing.
- Add failed-job handling to `run_processing()`.

Static maps or Street View images do not appear

Possible causes:

- Google API key does not have the required APIs enabled.
- Billing is not enabled on the Google Cloud project.
- API quota has been exceeded.
- The coordinate does not have Street View coverage.
- The generated file was not added to the ZIP because the row failed.

360-degree iframes do not load

Possible causes:

- Maps JavaScript API is not enabled.
- API key browser restrictions reject the current origin.
- The browser blocks the iframe content.
- The generated HTML contains an invalid or restricted key.

Development Notes

Quick syntax check

```
python -m compileall app.py maps2.py
```

Minimal mocked processing check

The processing function can be tested by monkeypatching the Google-dependent functions and passing a small dataframe.

Example concept:

```
import io
import pandas as pd
import maps2

maps2.reverse_geocode = lambda lat, lng: ("address", "place_id")
maps2.place_details = lambda place_id: {
    "formatted_address": "address",
    "name": "place",
    "types": ["type_a", "type_b"],
}
maps2.get_static_map = lambda lat, lng: io.BytesIO(b"map")
maps2.get_street_view = lambda lat, lng: None
maps2.get_360_html = lambda lat, lng: io.BytesIO(b"html")

rows, files = maps2.process_dataframe(
    pd.DataFrame([{"lat": 1.1, "lng": 2.2}])
)
```

Expected:

- One result row.
- `status` is `ok`.
- `map.png` and `360.html` are generated.

Maintenance Checklist

Before deploying or sharing the tool:

- Confirm `.env` is not committed.
- Confirm the Google API key is restricted.
- Disable Flask debug mode in production.
- Pin dependency versions if reproducible builds are required.
- Add upload size limits.
- Add coordinate range validation.
- Add failed-job state handling.
- Add cleanup for old jobs.
- Decide where generated files should be persisted.
- Fix template encoding artifacts.
- Update hard-coded footer year.